Akilesh87 opened 2w ago

Owner •••

Q1 **Smart Hospital Patient Registration System** 20 marl

Smart Hospital Patient Registration System

A multi-specialty hospital wants to digitize its outpatient registration system.

Every patient arriving at the hospital receives a registration number. The hospital management wants a Java application that maintains patient records efficiently while preventing duplicate registrations.

Each patient record consists of

- Patient ID
- Patient Name
- Mobile Number
- Department
- Doctor Name

The system should utilize Java Collection Framework and String manipulation techniques for storing, validating, searching, and displaying patient information.

Functional Requirements

Requirement 1 – Register Patient (4 Marks)

Accept

Patient ID

Patient Name

Mobile Number

Department

Doctor Name

Perform validations.

- Patient ID should be unique.

```java
            authors.add(author);

            System.out.println("Book Added Successfully.");

        } else if (choice == 2) {
            String id = trimSafe(sc.nextLine());
            Book b = bookById.get(id);

            if (b == null) {
                System.out.println("Book Not Available");
            } else {
                System.out.println("Book ID : " + b.id);
                System.out.println("Book Title : " + b.title);
                System.out.println("Author : " + b.author);
                System.out.println("Category : " + b.category);
                System.out.println("Publisher : " + b.publisher
            }

```

```java
            if (books.size() >= maxBooks) continue;

            String id = trimSafe(sc.nextLine());
            String title = trimSafe(sc.nextLine());
            String author = trimSafe(sc.nextLine());
            String category = trimSafe(sc.nextLine());
            String publisher = trimSafe(sc.nextLine());

            boolean ok = true;

            if (id.length() == 0) ok = false;
            if (title.length() == 0) ok = false;
            if (author.length() == 0) ok = false;
            if (category.length() == 0) ok = false;
            if (publisher.length() == 0) ok = false;
            if (bookById.containsKey(id)) ok = false;
```

```
                if (id.length() == 0) ok = false;
                if (title.length() == 0) ok = false;
                if (author.length() == 0) ok = false;
                if (category.length() == 0) ok = false;
                if (publisher.length() == 0) ok = false;
                if (bookById.containsKey(id)) ok = false;

                if (!ok) continue;

                title = toTitleCase(title);
                category = category.toUpperCase();

```

```
                out.append(ch);
            }
        }

        return out.toString().replaceAll(" +", " ");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        ArrayList<Book> books = new ArrayList<>();
        HashMap<String, Book> bookById = new HashMap<>();
        HashSet<String> authors = new HashSet<>();

        int maxBooks = 500;

        while (true) {
            if (!sc.hasNextLine()) break;
            String line = sc.nextLine().trim();
            if (line.length() == 0) continue;

            int choice;
```

```
    static String trimSafe(String s) {
        if (s == null) return "";
        return s.trim();
    }

    static String toTitleCase(String s) {
        s = trimSafe(s);
        if (s.length() == 0) return s;

        s = s.toLowerCase();
        StringBuilder out = new StringBuilder();
        boolean newWord = true;

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (ch == ' ') {
                out.append(' ');
                newWord = true;
            } else {
                if (newWord) {
                    if (ch >= 'a' && ch <= 'z') ch = (char) (ch - '
                    newWord = false;
```

```
import java.util.Scanner;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;

public class Main {

    static class Book {
        String id;
        String title;
```

Herbert Schildt

Mark Allen

Category-wise Count

PROGRAMMING : 1

COMPUTER SCIENCE : 1

Book Details

Book ID : B101

Title : Java Programming

Author : Herbert Schildt

Category : PROGRAMMING

Publisher : McGraw Hill

Constraints

Maximum books = 500

Duplicate Book IDs are not allowed.

Arrays are not allowed.

3

4

5

2

B101

6

Sample Output

Book Added Successfully.

Book Added Successfully.

Book ID Title Author

B101 Java Programming Herbert Schildt

B102 Data Structures Mark Allen

Unique Authors

ID TITLE AUTHOR CATEGORY

B101 Java Programming Herbert Schildt PROGRAMMING

B102 Data Structures Mark Allen COMPUTER SCIENCE

Requirement 4 – Display Unique Authors (3 Marks)

Display all distinct author names using HashSet.

Book Title should be stored in Title Case.

Category should be converted to uppercase.

Publisher name should not be empty.

Store the record in both

ArrayList

HashMap

Requirement 2 – Search Book (3 Marks)

ID TITLE AUTHOR CATEGORY

B101 Java Programming Herbert Schildt PROGRAMMING

B102 Data Structures Mark Allen COMPUTER SCIENCE

Requirement 4 – Display Unique Authors (3 Marks)

Display all distinct author names using HashSet.

Example

Unique Authors

Herbert Schildt

James Gosling

Yashavant Kanetkar

Mark Allen

Requirement 5 – Category-wise Book Count (4 Marks)

Display the number of books available in each category using

HashMap<String,Integer>

Example

Book Title should be stored in Title Case.

Category should be converted to uppercase.

Publisher name should not be empty.

Store the record in both

ArrayList

HashMap

Requirement 2 – Search Book (3 Marks)

Accept Book ID.

If found, display

Book ID

Book Title

Author

Category

Publisher

Otherwise,

Book Not Available

Requirement 3 – Display All Books (4 Marks)

Display all records in tabular format.

Menu Driven Program

1. Add Book
2. Search Book by ID
3. Display All Books
4. Display Unique Authors
5. Category-wise Book Count
6. Exit

Requirement 1 – Add Book (4 Marks)

```
} else {
    System.out.println("Patient Not Found");
}


:ic void displayAllPatients() {
System.out.println("----------------------------------------
System.out.println("ID\tName\tMobile\tDepartment\tDoctor");
System.out.println("----------------------------------------

for (String id : patientList) {
```

```java
String department = sc.nextLine().trim().toUpperCase();

String doctor = sc.nextLine().trim();

String[] patientData = {id, name, mobile, department, doctor};
patientMap.put(id, patientData);
patientList.add(id);

deptCount.put(department, deptCount.getOrDefault(department, 0) + 1

System.out.println("Patient Registered Successfully.");


ic void searchPatient(Scanner sc) {
System.out.print("");
String id = sc.nextLine().trim();

if (patientMap.containsKey(id)) {
    String[] p = patientMap.get(id);
    System.out.println("Patient Details");
    System.out.println("ID : " + p[0]);
    System.out.println("Name : " + p[1]);
```

```java
    static void registerPatient(Scanner sc) {
        if (patientMap.size() >= MAX_PATIENTS) {
            System.out.println("Maximum Patient Limit Reached.");
            return;
        }

        System.out.print("");
        String id = sc.nextLine().trim();

        if (patientMap.containsKey(id)) {
            System.out.println("Duplicate Patient ID. Registration
```

```java
    static void registerPatient(Scanner sc) {
        if (patientMap.size() >= MAX_PATIENTS) {
            System.out.println("Maximum Patient Limit Reached.");
            return;
        }

        System.out.print("");
        String id = sc.nextLine().trim();

        if (patientMap.containsKey(id)) {
            System.out.println("Duplicate Patient ID. Registration
            return;
        }

        String name = sc.nextLine().trim();
        if (!name.matches("[a-zA-Z ]+")) {
            System.out.println("Invalid Name. Only alphabets allowe
            return;
        }
        name = toProperCase(name);

        String mobile = sc.nextLine().trim();
```

```java
                case 1:
                    registerPatient(sc);
                    break;
                case 2:
                    searchPatient(sc);
                    break;
                case 3:
                    displayAllPatients();
                    break;
                case 4:
                    departmentWiseCount();
```

```java
import java.util.*;

public class Main {
    static ArrayList<String> patientList = new ArrayList<>();
    static HashMap<String, String[]> patientMap = new HashMap<>();
    static HashMap<String, Integer> deptCount = new HashMap<>();
    static final int MAX_PATIENTS = 100;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("1. Register Patient");
            System.out.println("2. Search Patient");
            System.out.println("3. Display All Patients");
            System.out.println("4. Count Department-wise Patients")
            System.out.println("5. Exit");

            choice = Integer.parseInt(sc.nextLine().trim());

            switch (choice) {
```

3

4

2

P101

5

Sample Output

Patient Registered Successfully

Sample Input

1

B101

Java Programming

Herbert Schildt

Programming

McGraw Hill

P101

5

Sample Output

Patient Registered Successfully.

Patient Registered Successfully.

ID Name Department

P101 Rahul Kumar CARDIOLOGY

P102 Priya ORTHO

Department Count

CARDIOLOGY : 1

Java Programming

Herbert Schildt

Programming

McGraw Hill

1

B102

Data Structures

Mark Allen

Computer Science

Pearson

3

Sample Input

1

P101

Rahul Kumar

9876543210

Cardiology

Dr.Arun

1

P102

Priya

9876543211